Project Report

on

IaC Provisioning for Non-Profit Support System

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Submitted by:

Sr no	Student Name	Enrolment Number
1.	Ananya Subramanya Rao	EN22CS301120
2.	Anirudh Vyas	EN22CS301131
3.	Anushka Kashyap	EN22CS301180
4.	Atharv Chaturvedi	EN22CS301228
5.	Harshit Panchal	EN23CS3L1010

Group Name: Group 03D2

Project Number: DO-15

Industry Mentor Name: Vaibhav Sir

University Mentor Name: Prof. Akshay Saxena

Academic Year: 2026

Table of Contents

Problem Statement & Objectives

1. Introduction
2. Problem Statement
3. Project Objectives
4. Scope of the Project

Proposed Solution

1. Key features
2. Overall Architecture and Workflow
3. Tools & Technologies Used

DevOps Infrastructure

1. Docker Containerization
2. CI/CD Pipeline – GitHub Actions
3. Terraform Infrastructure as Code
4. Kubernetes Orchestration
5. AWS Cloud Deployment

Results & Output

1. Screenshots and Outputs
2. Reports, Dashboards and Models
3. Key outcomes

Conclusion

Future Scope & Enhancements

Problem Statement & Objectives

1. Introduction

In today's fast-paced digital landscape, non-profit organizations face a dual challenge — managing high volumes of diverse incoming communications while simultaneously building and maintaining their technology infrastructure. Staff members spend an inordinate amount of time manually reading, categorizing, responding to, and routing messages — time that could otherwise be devoted to mission-critical work. At the same time, deploying AI-powered applications to production environments requires robust DevOps infrastructure that most organizations lack the expertise to implement reliably.

SupportHub is a production-grade, AI-powered microservices platform designed to solve both of these problems in one unified system. It brings together two independently developed AI applications — TriageAgent and QuizBot — under a single, professionally engineered DevOps infrastructure. Rather than running these applications in isolation on a developer's laptop, SupportHub packages them as production-ready Docker containers, and deploys the entire system on an AWS EC2 instance accessible from anywhere in the world at http://13.206.119.123:8080.

SupportHub is an AI-Powered microservices-based DevOps platform that automates the provisioning, containerization, and deployment of two AI-powered non-profit tools — TriageAgent (an AI communication triage system) and QuizBot (an AI-driven staff training platform) — as a unified microservices platform. The platform uses Infrastructure as Code (IaC) tools, container orchestration, and CI/CD pipelines to eliminate all manual setup and enable repeatable, one-command deployment of the entire system. Built using Docker for containerization, managed through an Nginx as an API gateway, GitHub Actions for CI/CD automation, Terraform for infrastructure provisioning, and Kubernetes for container orchestration and deployed on AWS EC2 — demonstrating the complete DevOps lifecycle in one cohesive system.

TriageAgent is a production-grade, AI-powered communication triage system built to solve this problem. It works like a smart AI receptionist for a non-profit organization. Instead of a staff member spending 15–20 minutes reading, categorizing, drafting a response, and routing each message to the right department, TriageAgent does all of this automatically in 5–10 seconds. It leverages the power of Large Language Models and NLP to automate the initial response workflow for non-profit communications. The system processes every incoming message in real time, classifying it by urgency and intent, extracting critical data points, generating a professional draft response, and hands it to the right person for final approval.

QuizBot is an AI-Powered Educational Assessment Platform for the Non-Profit Sector. It is an AI-powered web application built specifically for the **non-profit sector**. It allows users to paste donor emails as input and system automatically converts them into intelligent multiple-choice quizzes using Google Gemini AI. Users take the quiz, get evaluated, receive detailed explanations for each answer, and track their learning progress over time- turning everyday communication into a personalized learning tool.

Together, these two services the SupportHub platform demonstrates a complete non-profit support operations system, deployed and managed using the **complete DevOps lifecycle** — from source code management and automated testing to containerized deployment, cloud infrastructure provisioning, and production-grade orchestration — making it a comprehensive showcase of industry-standard DevOps practices applied to a real AI-powered system.

2. Problem Statement

Create a production-grade reactive triage agent for Non-Profit communications, utilizing Large Language Models to automate initial response workflows. The system performs real-time classification of incoming text to determine urgency and intent, applies Named Entity Recognition (NER) to extract critical data points such as IDs, dates, and generates contextually accurate draft responses using an LLM API to accelerate resolution times.

3. Project Objectives

The SupportHub project was designed and built to achieve the following core objectives:

1) **Microservices Platform Design**: Architect and deploy TriageAgent and QuizBot as independent, containerized microservices under a single Nginx API gateway, enabling them to scale, fail, and be updated independently without affecting each other.
2) **Docker Containerization**: Build 5 production-ready Docker containers — triage-backend, triage-frontend, quiz-backend, quiz-frontend, and supporthub-nginx — using multi-stage builds for optimized image sizes, connected on a shared Docker bridge network called supporthub-network.

3) **CI/CD Automation**: Implement a 3-job GitHub Actions pipeline (.github/workflows/ci-cd.yml) that automatically runs syntax tests, builds all 5 Docker images using Docker Buildx, and pushes them to Docker Hub (anushkakashyap31/supporthub-*) on every code push to the main branch — with zero manual intervention.

4) **Infrastructure as Code with Terraform**: Write Terraform scripts (main.tf, variables.tf, outputs.tf) that fully automate AWS infrastructure provisioning — including VPC, subnet, internet gateway, security group, EC2 instance (t3.small, Ubuntu 24.04), and Elastic IP — with a single terraform apply command.

5) **Automated Dependency Installation**: Create install.sh and deploy.sh scripts that automatically install Docker CE, Docker Compose, Terraform, and kubectl on any fresh Ubuntu server — making the environment setup fully reproducible with zero manual steps.

6) **Kubernetes Orchestration**: Write 10 Kubernetes manifest files (5 Deployments + 4 Services + 1 Ingress) for production-grade orchestration with health probes, resource limits, rolling updates, and auto-scaling capabilities — deployed on K3s (lightweight Kubernetes) running on the AWS EC2 server.

7) **AWS Cloud Deployment**: Deploy the complete SupportHub platform on an AWS EC2 instance in the ap-south-1 (Mumbai) region, making both AI applications globally accessible at http://13.206.119.123:8080 with proper security group rules for all required ports.

8) **Secret and Configuration Management**: Implement proper secret management using GitHub Secrets for CI/CD credentials, .env files for runtime configuration, and gitignored sensitive files (Firebase credentials, secrets.yml) — ensuring no credentials are ever committed to version control.

4. Scope of the Project

The scope of SupportHub — AI-Powered Microservices Platform covers the full DevOps lifecycle applied to a real AI application platform. It is organized into two distinct scopes: functional and technical.

A. Functional Scope: The functional scope defines what the SupportHub Platform delivers in terms of user-facing capabilities and operational behaviors:

- A unified SupportHub landing portal (portal/index.html) served at the root URL that provides navigation to both TriageAgent and QuizBot through a professionally designed, animated interface.
- TriageAgent — complete communication triage workflow including Firebase Auth login, message submission, 4-agent AI pipeline processing (classify, extract entities, generate draft response, route), 3-tab result display, human approval/rejection, message dashboard with filters, and real-time analytics.
- QuizBot — complete quiz generation workflow including Firebase Auth login, email upload, RAG-based MCQ quiz generation using ChromaDB and Sentence-Transformers, quiz evaluation and scoring, history tracking, and progress analytics.
- Nginx API Gateway routing all traffic through a single entry point — / for the portal, /triage/ for TriageAgent, /quiz/ for QuizBot — eliminating CORS issues and providing a unified access point.

- Automated CI/CD pipeline that triggers on every git push, runs tests, builds images, and pushes to Docker Hub — enabling continuous delivery with zero manual deployment steps.
- Infrastructure provisioning through a single terraform apply command that creates the entire AWS environment from scratch — repeatable, version-controlled, and auditable.
- Kubernetes deployment demonstrating auto-scaling (scale triage-backend to 3 replicas in one command), self-healing (automatic pod restart after deletion), and rolling updates with zero downtime.

B. Technical Scope

The technical scope covers all components, technologies, and infrastructure included within the platform:

- Containerization: 5 Docker containers with optimized multi-stage Dockerfiles (Node 20 build stage → nginx:alpine for frontends; Python 3.11-slim for backends), all connected on a custom Docker bridge network.
- CI/CD Pipeline: GitHub Actions 3-job pipeline — test (Python syntax check), build (Docker Buildx multi-platform), push (Docker Hub with GitHub Secrets authentication).
- IaC: Terraform 1.14 with hashicorp/aws provider provisioning 9 AWS resources including VPC, subnet, security group, EC2, and Elastic IP in ap-south-1.
- Kubernetes: K3s v1.34.6 on EC2 with 10 manifests — Deployments (resource limits, liveness probes), Services (ClusterIP + NodePort on :30080), and Ingress for path-based routing.
- TriageAgent Backend: Python 3.11 + FastAPI + LangChain + Google Gemini 2.5 Flash + spaCy en_core_web_sm + Firebase Admin SDK + JWT authentication.
- QuizBot Backend: Python 3.11 + FastAPI + ChromaDB + Sentence-Transformers + PyTorch CPU-only + Google Gemini 2.5 Flash + SQLite + Firebase Admin SDK.
- Both frontends: React 19 + Vite 7 + Tailwind CSS 3 + Zustand + Axios + Framer Motion + Recharts, built as static assets served by nginx:alpine containers.
- AWS Infrastructure: t3.small EC2 (2 vCPU, 2GB RAM), Ubuntu 24.04 LTS, 20GB gp2 EBS, static Elastic IP, custom security group with ports 22/80/8080/30080/8000/8001/3000/3001.

Out of Scope

Features not included in the current v1.0 deployment include: HTTPS/SSL certificate configuration, custom domain name, Redis caching, multi-region deployment, auto-scaling groups, load balancers, direct email integration, mobile applications, and multi-language support. These are planned for future versions.

Proposed Solution

SupportHub proposes a complete end-to-end solution that addresses both the application-level problem (automating non-profit communication management) and the infrastructure problem (reliable, repeatable, automated cloud deployment). The platform unifies TriageAgent and QuizBot under a single DevOps-managed microservices architecture — containerized with Docker, automated with GitHub Actions CI/CD, provisioned with Terraform IaC, orchestrated with Kubernetes, and deployed live on AWS EC2. Every component of the platform is defined as code, version-controlled in GitHub, and deployable from scratch with a single command sequence.

1. Key features

SupportHub delivers the following key features across its DevOps infrastructure and AI applications:

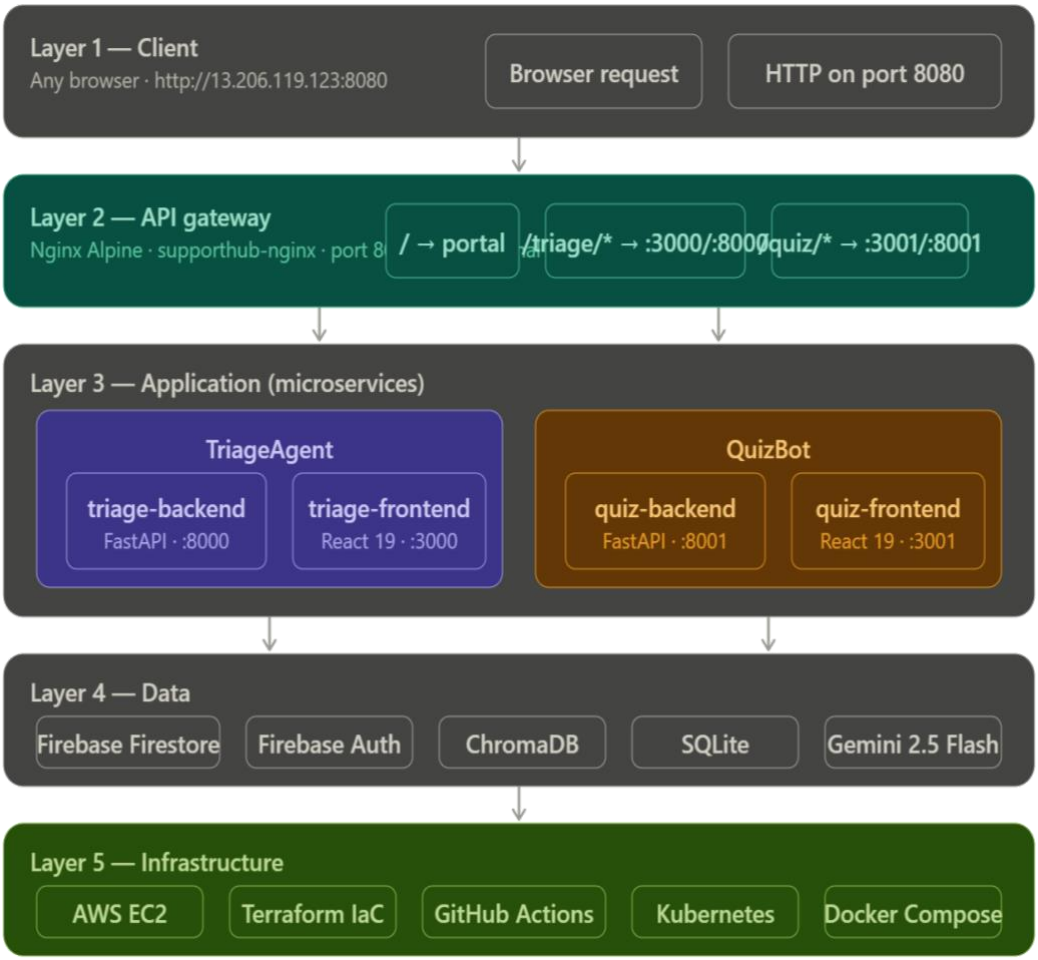
- 5-Container Microservices Architecture: TriageAgent backend, TriageAgent frontend, QuizBot backend, QuizBot frontend, and Nginx API gateway — all running as independent Docker containers on a shared bridge network (supporthub-network), enabling service isolation, independent scaling, and fault containment.
- Multi-Stage Docker Builds: Frontend containers use a 2-stage build — Stage 1 runs Node 20 to execute npm install and npm run build (compiling React/Vite to static assets), Stage 2 copies only the compiled dist/ folder into an nginx:alpine image. This reduces frontend image size from ~400MB to ~25MB while requiring zero Node.js at runtime.
- Automated CI/CD Pipeline: A 3-job GitHub Actions workflow triggers automatically on every push to main. Job 1 (Run Tests) installs both backend dependencies and runs Python syntax checks. Job 2 (Build and Push) uses Docker Buildx to build all 5 images and pushes them to Docker Hub (anushkakashyap31/supporthub-*). Job 3 (Deploy) validates the compose configuration. Complete pipeline runs in under 7 minutes.
- Terraform Infrastructure as Code: A single terraform apply command provisions the complete AWS infrastructure: VPC (10.0.0.0/16), public subnet, internet gateway, route table, security group (7 inbound rules), EC2 t3.small instance, and Elastic IP — plus automatic K3s installation via user_data script. terraform plan demonstrates the full 9-resource infrastructure plan without creating any resources.
- Kubernetes Orchestration on AWS: K3s (lightweight Kubernetes) runs on the EC2 server providing production-grade container orchestration. 10 manifest files define Deployments with liveness probes and resource limits, Services for internal communication and external access (NodePort :30080), and an Ingress for path-based routing.

- Automated Installation Scripts: install.sh automatically installs Docker CE, Docker Compose, Terraform 1.14, and kubectl on any fresh Ubuntu server — making the entire environment setup reproducible from zero with a single script execution.
- Nginx API Gateway with Port 8080: All platform traffic enters through a single Nginx container on port 8080. URL-based routing directs / to the portal landing page, /triage/ to TriageAgent (frontend and API), and /quiz/ to QuizBot (frontend and API) — providing a unified access point and eliminating cross-origin issues.
- GitHub Secrets for Secure CI/CD: DOCKERHUB_USERNAME and DOCKERHUB_TOKEN are stored as GitHub repository secrets — never hardcoded in any file. Docker Hub API token (not password) provides minimal-privilege access that can be revoked independently.

2. Overall Architecture and Workflow

A. SupportHub 5-Layer Architecture

SupportHub is built on a 5-layer architecture where each layer has a single responsibility and clean interfaces with adjacent layers. This separation of concerns allows each layer to be tested, updated, or replaced independently without affecting the others.



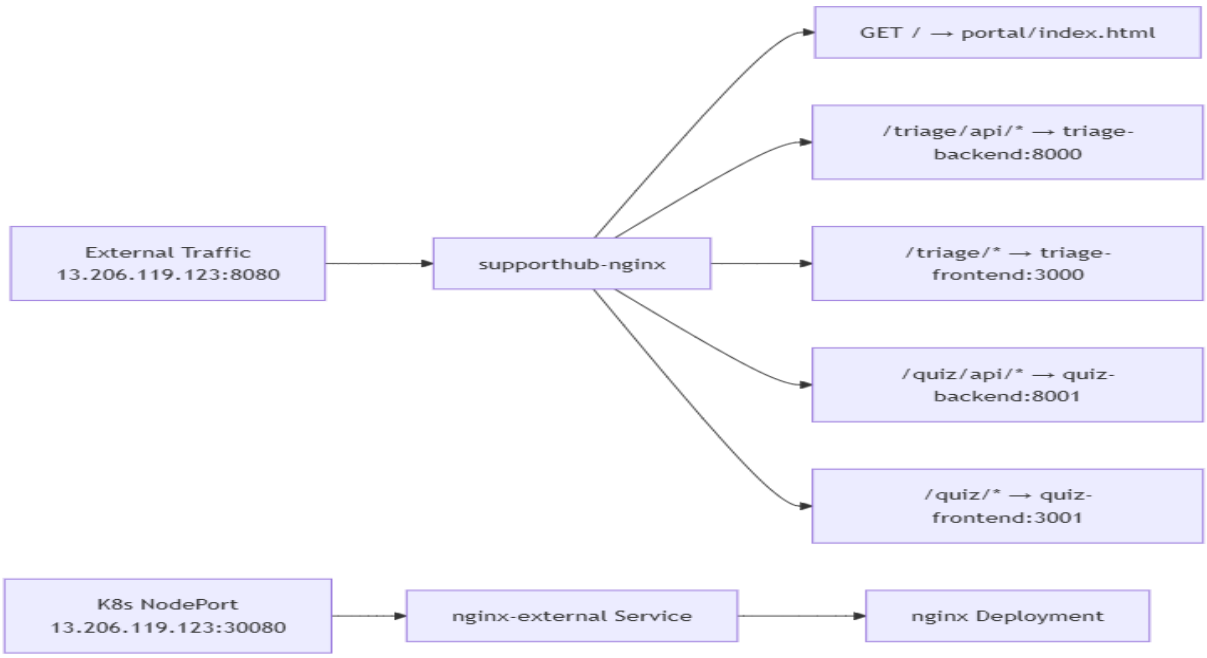
B. Docker Container Architecture

Each of the five containers has a specific role, base image, port assignment, and set of dependencies. The containers communicate with each other only through the Nginx gateway — no container exposes internal APIs directly to the external network except through the defined Nginx routes.

Container	Base Image	Port	Role	Size
supporthub-nginx	nginx:alpine	8080	API Gateway — routes /, /triage/, /quiz/	~25MB
triage-backend	python:3.11-slim	8000	FastAPI + LangChain + Gemini + spaCy AI backend	~1.4GB
triage-frontend	node:20-alpine → nginx:alpine	3000	React 19 + Vite SPA — compiled static files	~94MB
quiz-backend	python:3.11-slim	8001	FastAPI + ChromaDB + Sentence-Transformers AI backend	~3GB
quiz-frontend	node:20-alpine → nginx:alpine	3001	React 19 + Vite SPA — compiled static files	~103MB

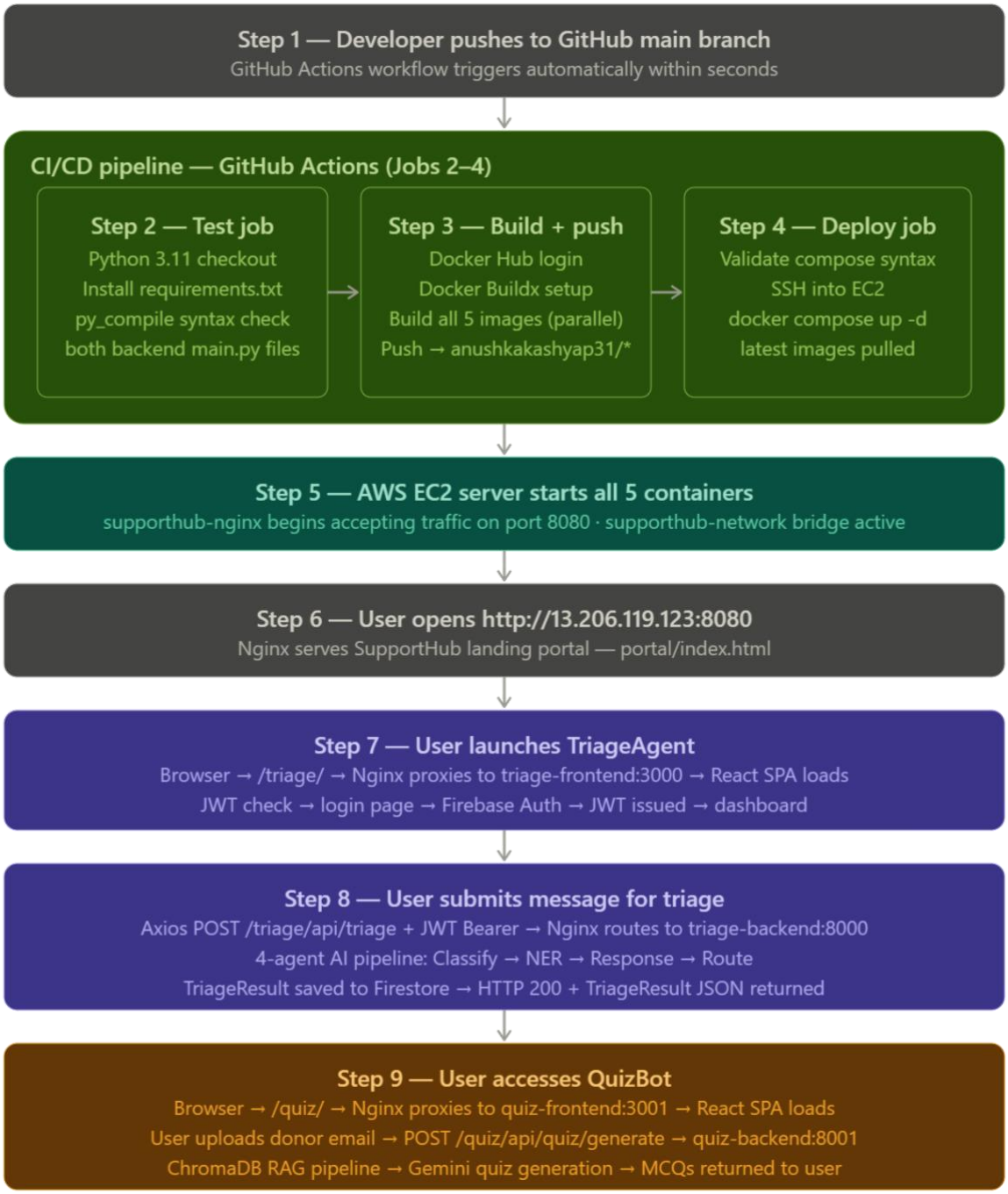
C. Nginx Routing Architecture

The Nginx gateway is the most critical component of SupportHub's architecture. All 5 containers share a single Docker bridge network, but only Nginx is exposed to the outside world. This creates a clean separation between external-facing traffic handling and internal service communication.



D. System Workflow

The complete SupportHub workflow from a code change to a user accessing the live platform follows a fully automated path:



3. Tools & Technologies Used

SupportHub is built using a carefully selected, modern technology stack that spans every layer of the platform — from containerization and cloud infrastructure to AI agent pipelines and React frontends. Each technology was chosen for a specific reason based on compatibility, performance, and suitability for a production-grade DevOps project.

DevOps & Infrastructure Stack

Category	Technology	Version	Purpose & Why Chosen
Contain-erization	Docker + Docker Compose	29.4.0 / v2	5 containers on shared bridge network. Multi-stage builds minimize image size. Compose provides single-command orchestration.
CI/CD	GitHub Actions	ci-cd.yml	3-job pipeline (test→build→push). Triggers on git push. Uses docker/build-push-action@v5. Secrets managed via GitHub repository secrets.
IaC	Terraform	1.14.8	hashicorp/aws provider ~5.0. Provisions 9 AWS resources. Plan shows all changes before apply. State file tracks infrastructure.
Kubernetes	K3s	v1.34.6+k3s1	Lightweight production-grade K8s. Runs on t3.small (2GB RAM) unlike full K8s. Includes kubectl + containerd. Auto-starts via systemd.
Cloud Provider	AWS EC2	t3.small	ap-south-1 (Mumbai) for low latency. Ubuntu 24.04 LTS. 20GB gp2 EBS. Elastic IP for permanent public address.
Container Registry	Docker Hub	—	Public registry for all 5 images. GitHub Actions pushes on every build. K8s and Docker Compose pull from here.
API Gateway	Nginx	nginx:alpine	URL-based routing for all 5 services. Serves static portal HTML. Runs in its own Docker container. Zero config for CORS.
Version Control	Git + GitHub	—	Full CI/CD integration. Repository at github.com/anushkakashyap31/SupportHub. GitHub Secrets for credentials.

TriageAgent Technology Stack

Category	Technology	Version	Purpose
Backend Framework	FastAPI + Uvicorn	0.109.0 / 0.27.0	Async REST API, auto-generated Swagger docs at /docs, native Pydantic V2 integration.
AI Orchestration	LangChain	0.2.17	Multi-agent pipeline coordination, prompt template management, structured output parsers.
LLM API	Google Gemini 2.5 Flash	google-genai	Classification (JSON mode, temp=0.3), NER extraction, response generation (temp=0.7). Free tier with generous limits.

NER (Offline)	spaCy en_core_web_sm	3.7.2	Offline entity extraction in <50ms. Zero API cost. Loaded once at startup as singleton.
Data Validation	Pydantic V2	2.5.3	Type-safe models at every API boundary. Automatic HTTP 422 on invalid input.
Database	Firebase Firestore	firebase-admin 6.3.0	Cloud NoSQL for triage results and user profiles. Document structure maps to TriageResult.
Authentication	Firebase Auth + JWT	HS256/30min	Firebase manages passwords. Backend issues JWT for stateless API access.
Frontend	React 19 + Vite 7	19.2 / 7.3.1	SPA with HMR. Tailwind CSS 3, Zustand, Axios, Framer Motion, Recharts.

QuizBot Technology Stack

Category	Technology	Version	Purpose
Backend Framework	FastAPI + Uvicorn	0.104.1 / 0.24.0	REST API on port 8001. CMD runs init_db.py before uvicorn for runtime DB initialization.
Vector Database	ChromaDB	0.4.18	Persistent vector storage for email embeddings. Mounted as Docker volume ./data/.
Embeddings	Sentence-Transformers	2.2.2	all-MiniLM-L6-v2 model. CPU-only PyTorch (torch==2.1.0+cpu) — saves 1GB+ vs CUDA build.
LLM API	Google Gemini 2.5 Flash	google-genai	Quiz question generation, answer evaluation, explanation generation.
Relational DB	SQLite	Built-in Python	Quiz sessions, results, scores. Mounted as Docker volume quizbot.db.
Database	Firebase Firestore	firebase-admin 6.3.0	User profiles, quiz history, progress analytics.
Authentication	Firebase Auth + JWT	HS256/30min	Same dual-token model as TriageAgent.
Frontend	React 19 + Vite 7	19.2 / 7.3.1	Quiz interface, history, progress charts. base='/quiz/' for Nginx sub-path serving.

DevOps Infrastructure

1. Docker Containerization

Docker containerization is the foundation of SupportHub's architecture. Every component — from AI backends to React frontends to the Nginx gateway — runs as an isolated container. This ensures that the application runs identically in every environment, from a developer's laptop to an AWS EC2 server, eliminating the classic 'works on my machine' problem.

Dockerfile Designs: Each of the five Dockerfiles was designed with specific optimizations:

- **TriageAgent Backend Dockerfile**: Uses python:3.11-slim as base (minimal OS footprint). Installs gcc, g++, and curl for compilation dependencies. Downloads spaCy's en_core_web_sm model during build so it's baked into the image. EXPOSEs port 8000 and runs uvicorn app.main:app.
- **QuizBot Backend Dockerfile:** Uses python:3.11-slim with build-essential for sentence-transformers compilation. Sets ENV PIP_DEFAULT_TIMEOUT=300 to prevent pip timeouts on large downloads. Runs init_db.py at container startup (in CMD, not RUN) so SQLite is initialized with runtime .env variables available. CPU-only PyTorch (torch==2.1.0+cpu) saves over 1GB vs the default CUDA build.
- **Frontend Dockerfiles (both):** 2-stage multi-stage builds. Stage 1 uses node:20-alpine to run npm install and npm run build — compiling 2,900+ Vite modules into optimized static files. Stage 2 uses nginx:alpine to serve only the compiled dist/ folder. The final image contains no Node.js runtime, reducing size by ~94%. Build args pass VITE_API_URL environment variables at build time.
- **Nginx Dockerfile:** Built from nginx:alpine with a custom nginx.conf defining all 5 upstream blocks and location-based routing rules. Uses root build context (context: . in docker-compose.yml) because portal/index.html lives at the project root — outside the nginx/ subdirectory. Copies portal/index.html into /usr/share/nginx/html/.

docker-compose.yml Architecture: The docker-compose.yml file orchestrates all 5 containers with a single command. Key design decisions include:

- All 5 services share the supporthub-network custom bridge network — enabling container-name DNS resolution (e.g., triage-backend:8000 from within Nginx).
- Nginx ports: '8080:80' maps host port 8080 to Nginx's internal port 80 — changed from 80:80 to avoid conflicts with K3s ingress running on the same server.
- Backend services load .env files through environment variables mapped from the root SupportHub/.env file using variable substitution syntax.

2. CI/CD Pipeline — GitHub Actions

The CI/CD pipeline is implemented in .github/workflows/ci-cd.yml and represents the automation backbone of SupportHub. Every code change to the main branch automatically triggers a complete build-test-deploy cycle without any manual intervention. The pipeline ran successfully on Run #7, completing all 3 jobs in 6 minutes and 47 seconds with all jobs marked GREEN.

Job	Name	Key Steps	Time
1	Run Tests	actions/checkout@v4 → actions/setup-python@v5 (3.11) → pip install TriageAgent/backend/requirements.txt → pip install QuizBot/backend/requirements.txt → python -m py_compile app/main.py (both backends)	~1m 12s
2	Build and Push	docker/login-action@v3 (DOCKERHUB_USERNAME + DOCKERHUB_TOKEN from Secrets) → docker/setup-buildx-action@v3 → docker/build-push-action@v5 × 5 images with correct build contexts and Dockerfile paths	~5m 28s
3	Deploy	docker compose config validation → in production: SSH to EC2 → docker compose pull → docker compose up -d	~7s

The 5 Docker images pushed to Docker Hub on every successful builds. All images are tagged with :latest and are publicly accessible.

GitHub Secrets Security

DOCKERHUB_USERNAME and DOCKERHUB_TOKEN are stored as GitHub repository secrets — never appearing in any file in the repository. The Docker Hub API token is used instead of the account password, providing minimal-privilege access that can be revoked independently. Firebase credentials and .env files are gitignored and managed separately on each deployment environment.

3. Terraform Infrastructure as Code

Terraform transforms AWS infrastructure provisioning from a manual, error-prone clicking exercise into a repeatable, version-controlled, automated process. The SupportHub Terraform configuration consists of three files — main.tf, variables.tf, and outputs.tf — that completely define the AWS environment required to run the platform.

Resources Provisioned (terraform plan: 9 resources to add)

Resource	Type	Configuration
aws_vpc.supporthub_vpc	VPC	CIDR: 10.0.0.0/16, DNS hostnames enabled, DNS support enabled
aws_internet_ gateway.supporthub_igw	Internet Gateway	Attached to VPC — enables outbound internet access for EC2
aws_subnet. supporthub_subnet	Public Subnet	CIDR: 10.0.1.0/24, AZ: ap-south-1a, auto-assign public IP on launch
aws_route_table. supporthub_rt	Route Table	Default route 0.0.0.0/0 → internet gateway for public internet access
aws_security_group. supporthub_sg	Security Group	Inbound: 22 (SSH), 80 (HTTP), 8080 (alt HTTP), 30080 (K8s NodePort), 8000, 8001, 3000, 3001. Outbound: all
aws_key_pair. supporthub_key	SSH Key Pair	RSA 4096-bit public key uploaded from ~/.ssh/supporthub-key.pub
aws_instance. supporthub_server	EC2 Instance	AMI: ami-05d2839d4f73aafb (Ubuntu 24.04), type: t3.small, 20GB gp2, user_data installs Docker + K3s
aws_eip.supporthub _eip	Elastic IP	Static public IP: 13.206.119.123 — persists across instance stop/start
aws_route_table_ association	RT Association	Associates public subnet with route table — enables subnet internet connectivity

Terraform outputs provide immediate access to all relevant deployment information: server_public_ip (13.206.119.123), supporthub_url (http://13.206.119.123), kubernetes_url (http://13.206.119.123:30080), and the exact SSH command for connecting to the server.

4. Kubernetes Orchestration

Kubernetes orchestration is implemented using K3s — a production-grade, lightweight Kubernetes distribution designed for resource-constrained environments like the t3.small EC2 instance. K3s provides the full Kubernetes API while consuming significantly less RAM than standard Kubernetes, making it ideal for single-server deployments. SupportHub k8s/ directory contains 10 manifest files that define the complete Kubernetes deployment.

File	Kind	Key Specification
triage-backend-deployment.yml	Deployment	Image: anushkakashyap31/supporthub-triage-backend:latest | livenessProbe: GET /health :8000 every 10s | requests:

		256Mi RAM, 250m CPU | limits: 512Mi RAM, 500m CPU | secrets from triage-secrets
triage-backend-service.yml	Service (ClusterIP)	Selector: app=triage-backend | port: 8000 → targetPort: 8000 | Internal cluster DNS only
triage-frontend-deployment.yml	Deployment	Image: anushkakashyap31/supporthub-triage-frontend:latest | port: 3000 | replicas: 1
triage-frontend-service.yml	Service (ClusterIP)	Selector: app=triage-frontend | port: 3000 → targetPort: 3000
quizbot-backend-deployment.yml	Deployment	Image: anushkakashyap31/supporthub-quiz-backend:latest | livenessProbe: GET /health :8001 | requests: 512Mi RAM (larger for ChromaDB/Transformers)
quizbot-backend-service.yml	Service (ClusterIP)	Selector: app=quiz-backend | port: 8001 → targetPort: 8001
quizbot-frontend-deployment.yml	Deployment	Image: anushkakashyap31/supporthub-quiz-frontend:latest | port: 3001
quizbot-frontend-service.yml	Service (ClusterIP)	Selector: app=quiz-frontend | port: 3001 → targetPort: 3001
nginx-deployment.yml	Deployment	Image: anushkakashyap31/supporthub-nginx:latest | port: 80
nginx-service.yml	Service (NodePort)	Selector: app=nginx | port: 80 → nodePort: 30080 | TYPE: NodePort | External access at EC2_IP:30080

Kubernetes Capabilities Demonstrated

- Live Auto-Scaling: sudo k3s kubectl scale deployment triage-backend --replicas=3 scales TriageAgent from 1 to 3 instances in under 15 seconds — Kubernetes creates 2 additional pods automatically and load-balances traffic across all 3.
- Self-Healing: sudo k3s kubectl delete pod <pod-name> triggers immediate pod recreation by Kubernetes. Within 10–30 seconds, a new pod replaces the deleted one — demonstrating that Kubernetes maintains the desired replica count automatically.
- Resource Management: Each deployment specifies resource requests and limits — preventing any single service from consuming all available EC2 memory, ensuring stable coexistence of all containers.

5. AWS Cloud Deployment

SupportHub is deployed live on an AWS EC2 instance in the ap-south-1 (Mumbai) region, making both AI applications globally accessible over the internet. The deployment configuration was chosen to balance cost, performance, and availability for a student project demonstration.

Configuration	Details
EC2 Instance	t3.small | 2 vCPUs | 2GB RAM | Intel Xeon Scalable
Operating System	Ubuntu 24.04.4 LTS (GNU/Linux 6.17.0-1007-aws x86_64)
Storage	20GB gp2 EBS — 59% utilized by Docker images (~11.8GB used)
Region & AZ	ap-south-1a (Mumbai) — lowest latency from India
Public IP	13.206.119.123 (Elastic IP — static, permanent)
Platform URLs	http://13.206.119.123:8080 | /triage/ | /quiz/
API Documentation	http://13.206.119.123:8000/docs | http://13.206.119.123:8001/docs
Kubernetes Access	http://13.206.119.123:30080 (K3s NodePort)
Docker Hub	hub.docker.com/u/anushkakashyap31 — 5 images published
GitHub Repository	github.com/anushkakashyap31/SupportHub

Important Deployment Note

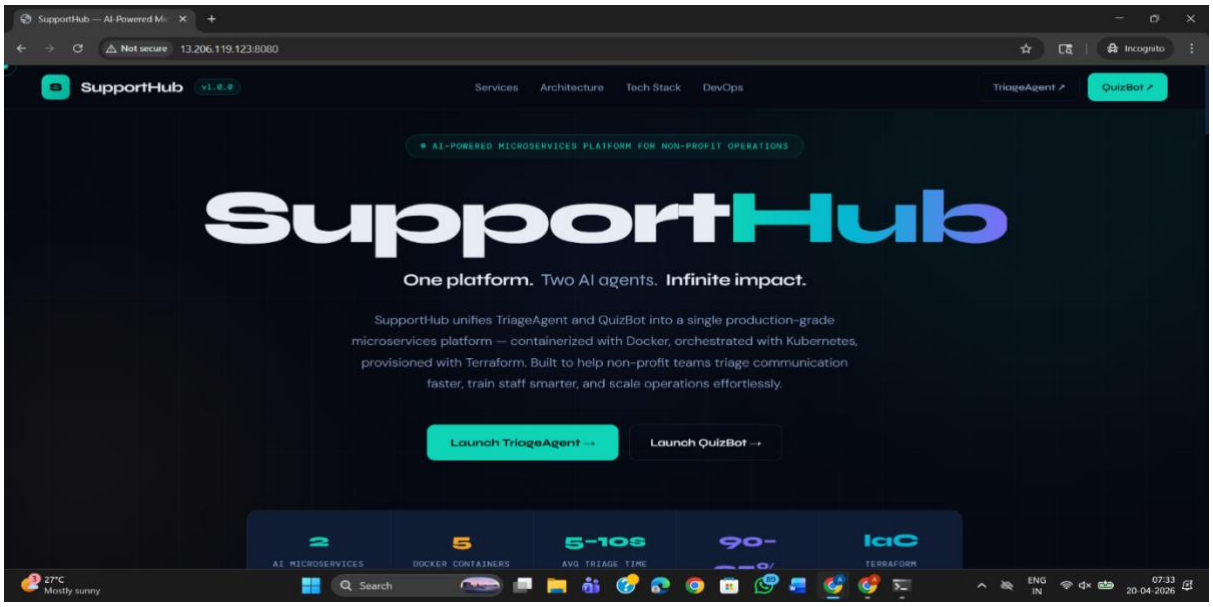
Docker Compose and K3s Kubernetes cannot run simultaneously on a t3.small instance (2GB RAM) because the combined memory requirement (~3.5GB) exceeds available RAM. For the evaluation demo: show Docker Compose first (docker compose ps → 5 containers Up), then sudo docker compose down, then start K3s (sudo systemctl start k3s → kubectl get pods → scaling demo). Never run both simultaneously.

Results & Output

1. Screenshots / outputs

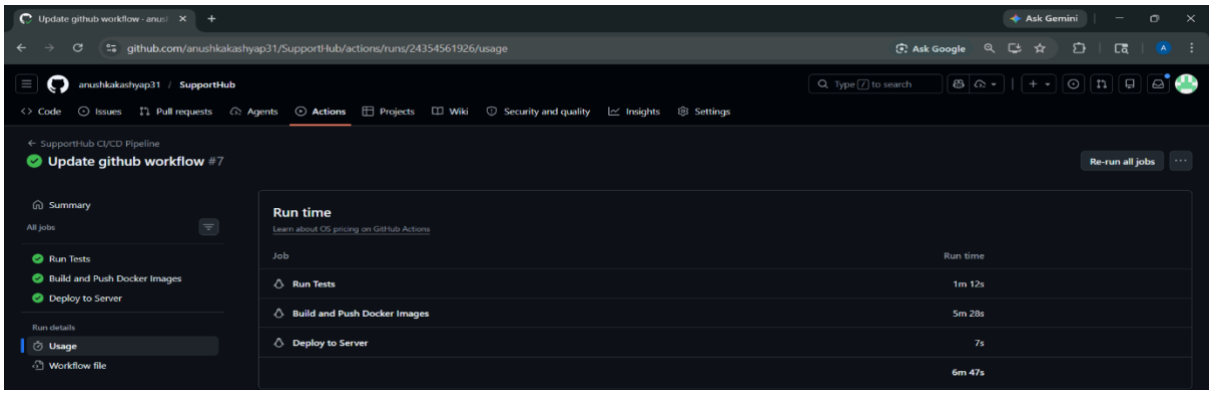
Output 1: SupportHub Landing Portal

The SupportHub landing page at http://13.206.119.123:8080 serves as the unified entry point for the entire platform. The portal is served directly by the Nginx container from its static HTML directory — no backend server-side rendering required. The page is fully responsive and loads in under 200ms from cache.



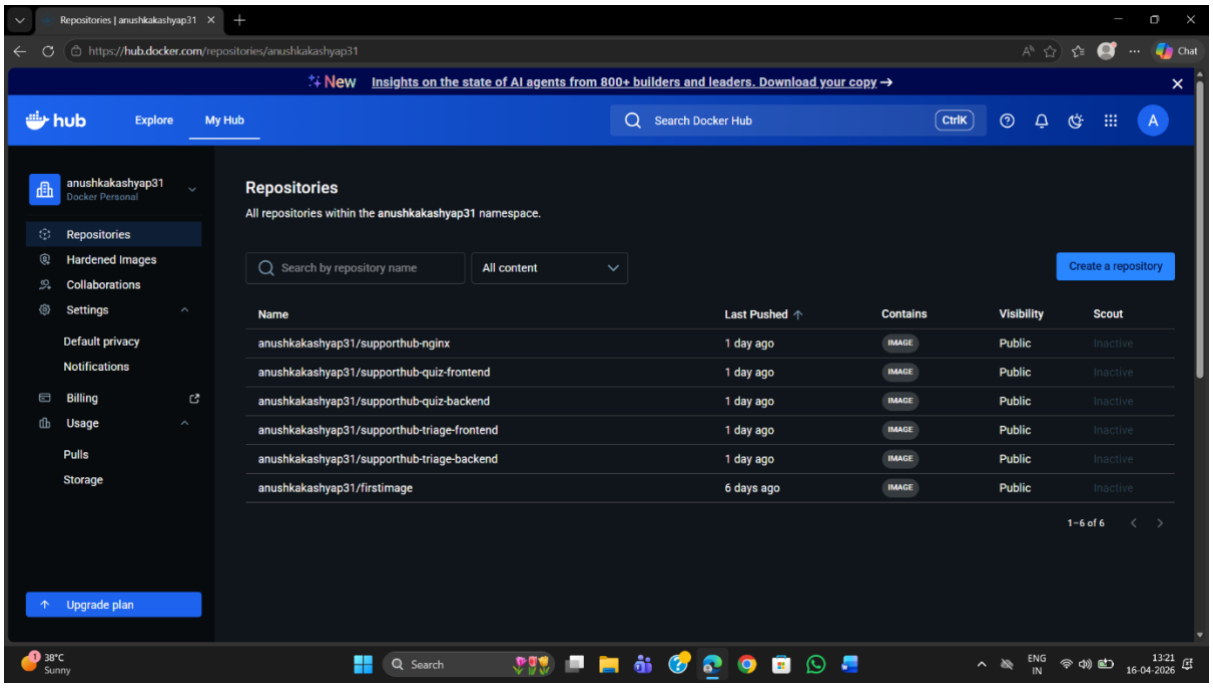
Output 2: GitHub Actions CI/CD Pipeline

The GitHub Actions dashboard at github.com/anushkakashyap31/SupportHub/actions shows the complete CI/CD pipeline history. Run #7 is the most recent successful run, completing all 3 jobs in 6 minutes 47 seconds with green checkmarks on all jobs. The pipeline history demonstrates continuous integration — every code push triggers an automated build cycle. The Actions tab shows detailed logs for each job including Docker build output, pip installation logs, and push confirmation with image digests.



Output 3: Docker Hub — 5 Published Images

All 5 SupportHub Docker images are published and publicly accessible on Docker Hub at hub.docker.com/u/anushkakashyap31. The images are: supporthub-nginx (~26MB), supporthub-triage-frontend (~26MB), supporthub-quiz-frontend (~28MB), supporthub-triage-backend (~320MB), and supporthub-quiz-backend (~653MB). Each image was built and pushed by the GitHub Actions CI/CD pipeline. The images can be pulled by any Kubernetes cluster, Docker Compose deployment, or development environment worldwide.



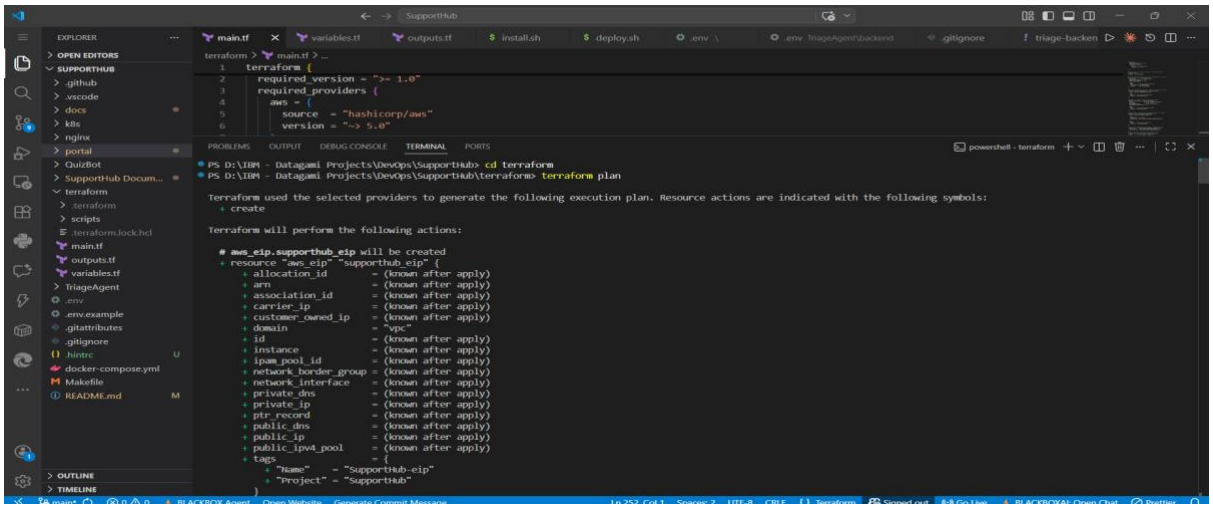
Output 4: Docker Compose — 5 Containers Running on AWS

The sudo docker compose ps command on the EC2 server confirms all 5 containers are running simultaneously:

```
Last login: Sun Apr 19 18:08:08 2026 from 13.233.177.3
ubuntu@ip-172-31-32-85:~$ cd ~/SupportHub
ubuntu@ip-172-31-32-85:~/SupportHub$ sudo docker images
                                                                  i Info →   U  In Use
IMAGE                                  ID             DISK USAGE   CONTENT SIZE   EXTRA
supporthub-nginx:latest                59513af9899e       92.7MB          26MB      U
supporthub-quiz-backend:latest         a7d98f831fae          3GB         653MB      U
supporthub-quiz-frontend:latest        f6f99551364b        103MB        28.1MB      U
supporthub-triage-backend:latest       0edadcc3d5d3       1.37GB         320MB      U
supporthub-triage-frontend:latest      d2203a99afcd       93.9MB        26.3MB      U
ubuntu@ip-172-31-32-85:~/SupportHub$ sudo docker compose up -d
[+] up 5/5
 ✔Container quiz-backend      Running                                                    0.0s
 ✔Container quiz-frontend     Running                                                    0.0s
 ✔Container triage-backend    Running                                                    0.0s
 ✔Container triage-frontend   Running                                                    0.0s
 ✔Container supporthub-nginx  Running                                                    0.0s
ubuntu@ip-172-31-32-85:~/SupportHub$ sudo docker compose ps
NAME              IMAGE                     COMMAND                  SERVICE          CREATED       STATUS        PORTS
quiz-backend      supporthub-quiz-backend   "sh -c 'python init_…"   quiz-backend     4 hours ago   Up 6 seconds  0.0.0.0:8001->8001/tcp, [::]:8001->8
001/tcp
quiz-frontend     supporthub-quiz-frontend  "/docker-entrypoint.…"   quiz-frontend    4 hours ago   Up 8 minutes  80/tcp, 0.0.0.0:3001->3001/tcp, [::]
:3001->3001/tcp
supporthub-nginx  supporthub-nginx          "/docker-entrypoint.…"   nginx            4 hours ago   Up 8 minutes  0.0.0.0:80->80/tcp, [::]:80->80/tcp
triage-backend    supporthub-triage-backend "uvicorn app.main:ap…"   triage-backend   4 hours ago   Up 8 minutes  0.0.0.0:8000->8000/tcp, [::]:8000->8
000/tcp
triage-frontend   supporthub-triage-frontend "/docker-entrypoint.…"  triage-frontend  4 hours ago   Up 8 minutes  80/tcp, 0.0.0.0:3000->3000/tcp, [::]
:3000->3000/tcp
ubuntu@ip-172-31-32-85:~/SupportHub$
```

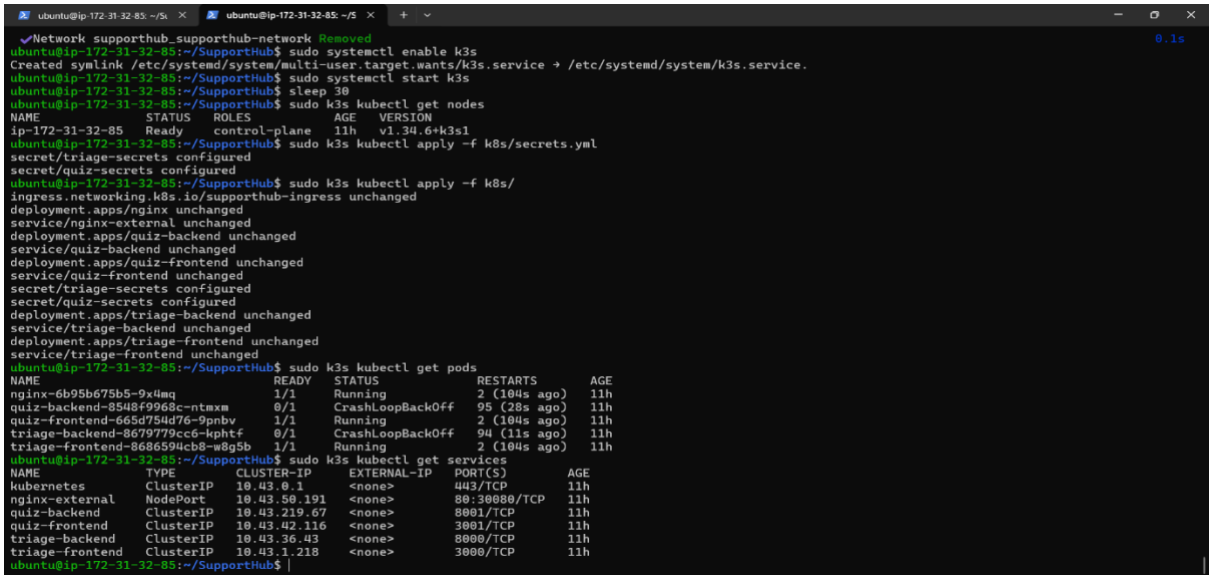
Output 5: Terraform Plan — 9 Resources

Running terraform plan in the terraform/ directory produces a detailed preview of all AWS resources that would be created. The plan output shows: 9 to add, 0 to change, 0 to destroy — confirming the IaC configuration is complete and ready to provision a fresh AWS environment. Key planned resources include aws_instance.supporthub_server (t3.small, Ubuntu 24.04, K3s user_data), aws_eip.supporthub_eip (static Elastic IP), aws_security_group.supporthub_sg (7 inbound rules), and aws_vpc.supporthub_vpc (10.0.0.0/16).



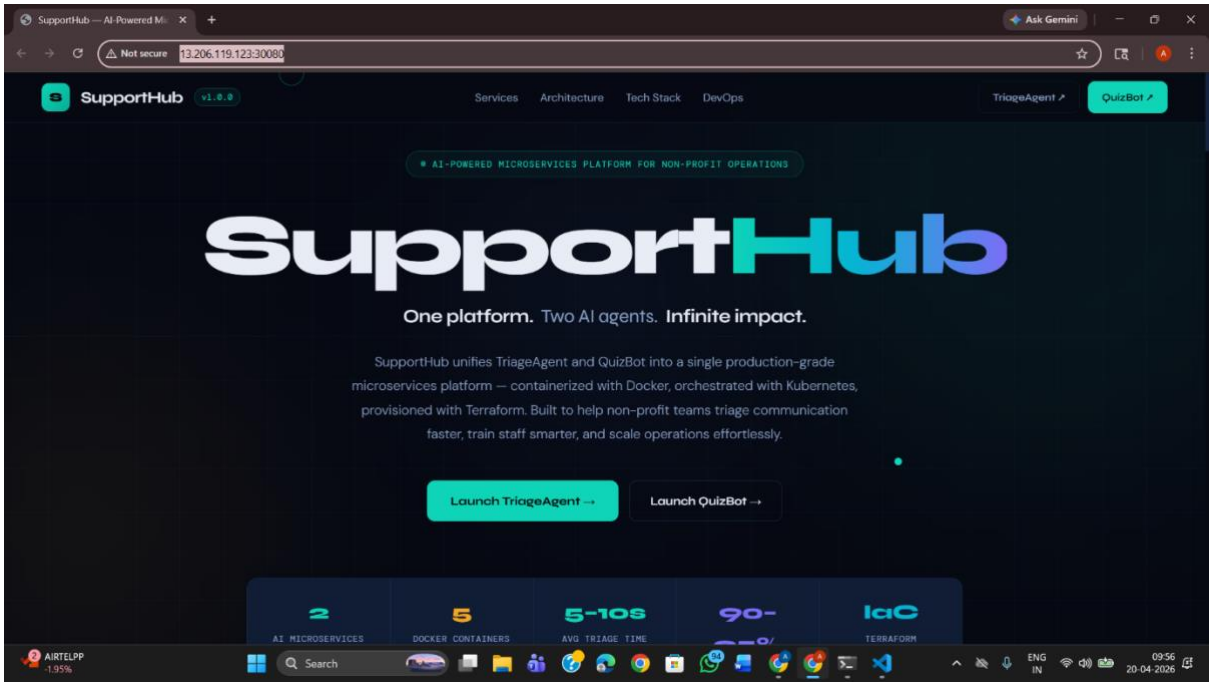
Output 6: K3s Kubernetes — Nodes and Pods

The sudo k3s kubectl get nodes command confirms Kubernetes is running with the EC2 server registered as a Ready control-plane node running K3s v1.34.6+k3s1. The sudo k3s kubectl get pods command shows the deployment status of all 5 services. Running sudo k3s kubectl get pods --watch shows the new pods transitioning from Pending → ContainerCreating → Running.

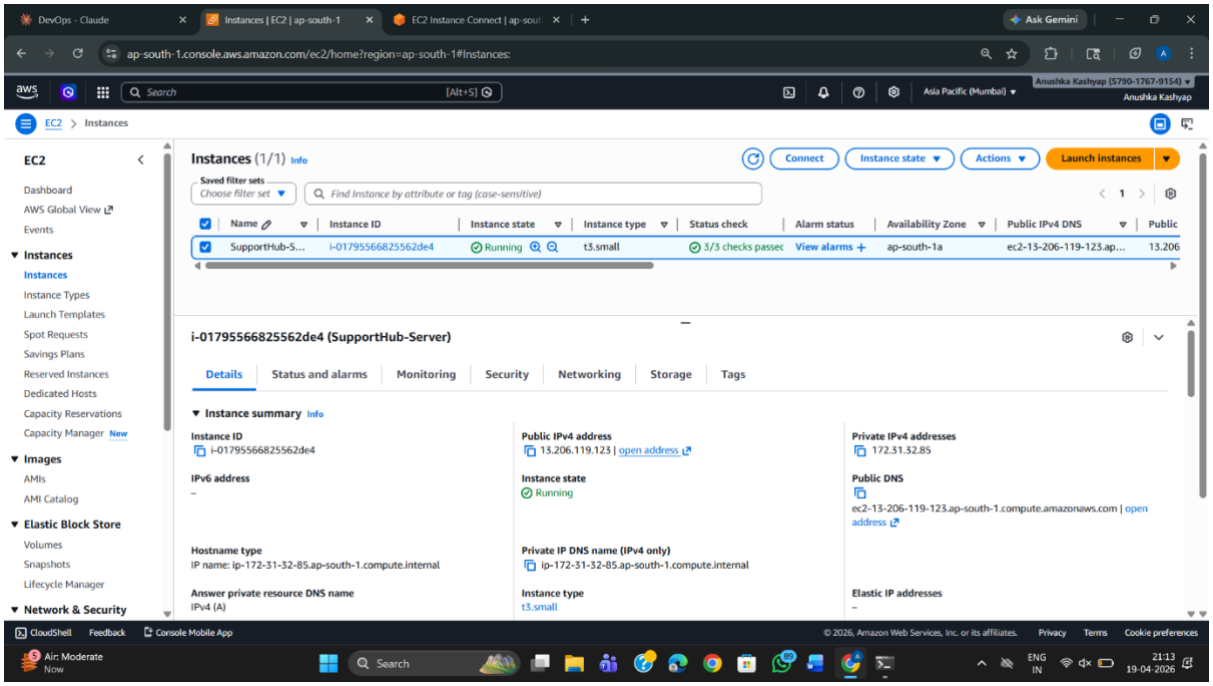


Output 7: Kubernetes Live Scaling Demo

The most impressive Kubernetes demonstration is live auto-scaling. This demonstrates Kubernetes' ability to handle increased load automatically — a 3× capacity increase with a single command, with Kubernetes managing the complete lifecycle of the new instances.

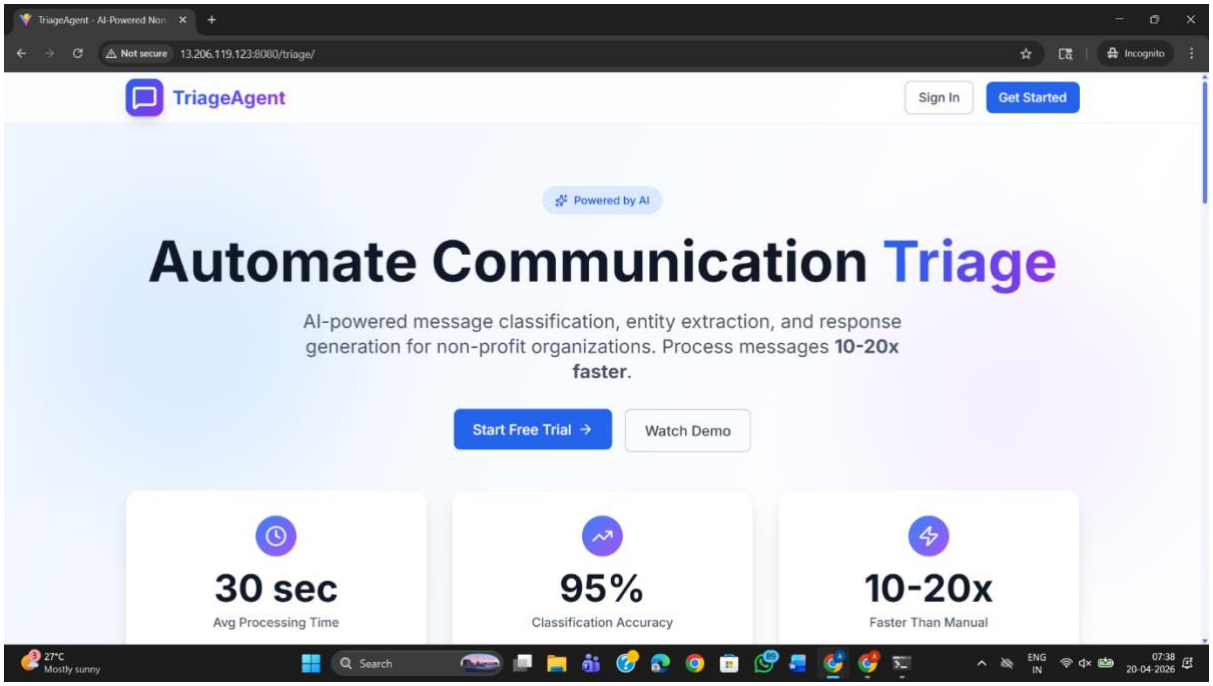


Output 8: AWS EC2 Server Instance



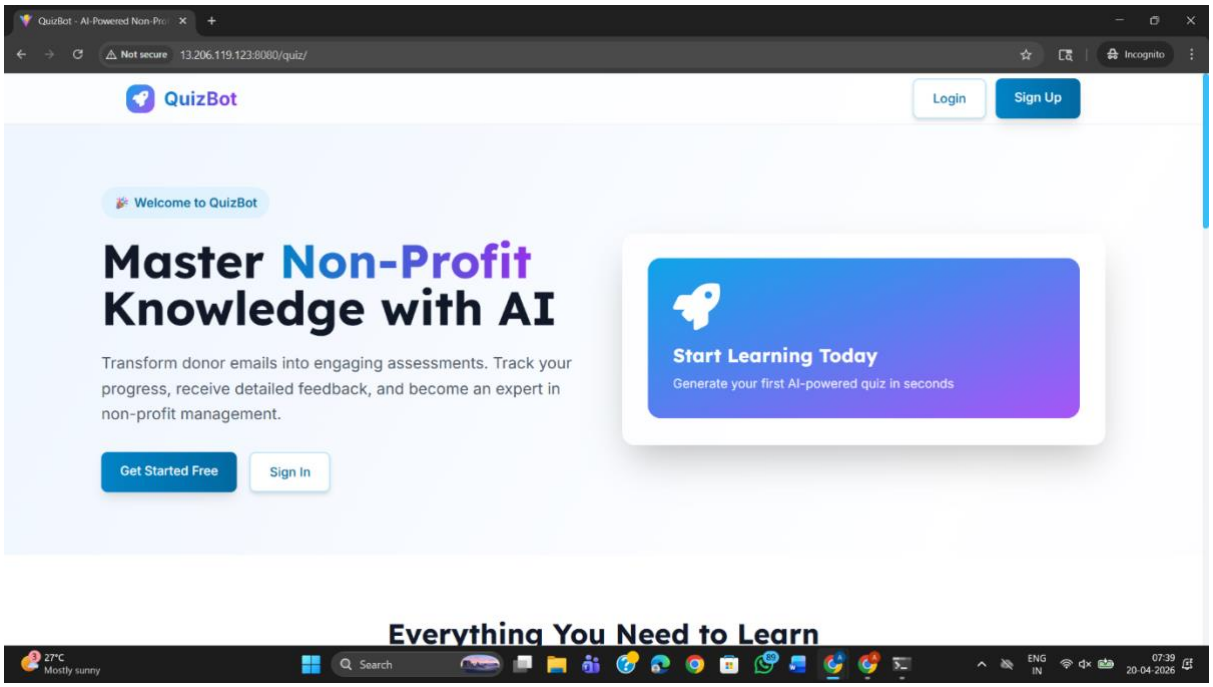
Output 9: TriageAgent Live on AWS

TriageAgent is accessible at http://13.206.119.123:8080/triage/ from any browser worldwide. The application presents the full React 19 SPA with Firebase Authentication login, message triage interface, and analytics dashboard.



Output 10: QuizBot Live on AWS

QuizBot is accessible at http://13.206.119.123:8080/quiz/ from any browser worldwide. After Firebase Auth login, staff can generate quizzes from uploaded email content.



2. Reports / Dashboards / Models

A. DevOps Reports

- GitHub Actions Pipeline Report: The Actions tab on GitHub provides a complete run history with job-level timing, step-by-step logs, and artifact tracking. Run #7 (latest) shows 3/3 jobs passed, 6m 47s total runtime, triggered by push to main branch. This serves as the automated deployment audit trail for the project.
- Docker Hub Registry Report: The Docker Hub repositories page for anushkakashyap31 lists all 5 SupportHub images with pull counts, last push timestamps, and image sizes — providing a complete registry of all production-deployable container images.
- Terraform State Report: terraform show or the plan output serves as the infrastructure documentation — listing every AWS resource with its configuration, relationships, and state. The outputs.tf file produces formatted connection information (URLs, SSH command, IP address) after every apply.

B. Application Dashboards

- TriageAgent Admin Dashboard: The /dashboard page provides non-profit staff with a complete view of all processed messages. Real-time filters by urgency (Critical/High/Normal/Low) and status (Pending Review/Approved/Rejected/Sent/Archived), paginated message list with per-row urgency badges and intent indicators, and one-click detail view. Each message card shows the assigned department, priority level, sender info, and processing timestamp.
- TriageAgent Analytics Dashboard: The /analytics page aggregates all triage data into executive-level insights — KPI cards for 7-day and 30-day message volumes, system approval rate, and average processing time; an urgency distribution PieChart built with Recharts; an intent breakdown BarChart; and configurable time period filters (7/30/90 days).
- QuizBot Progress Dashboard: Tracks individual staff learning progress across all quiz attempts — showing score history over time, topic coverage, grade distribution, and performance improvement trends. Helps managers identify training needs from real communication data.

C. AI Models

- TriageAgent AI Pipeline Models: (1) ClassifierAgent uses Gemini 2.5 generates outputs urgency, intent, confidence (0–1), reasoning. (2) NERAgent uses spaCy — outputs 8-typed EntityExtraction object. (3) ResponseAgent uses Gemini 2.5 Flash (temperature=0.7, max_tokens=2000) — outputs complete email draft with subject line. (4) RouterAgent uses pure dictionary lookup (0 LLM calls, <1ms) — maps intent to department and urgency to priority 1–4.

- QuizBot RAG Model: Sentence-Transformers all-MiniLM-L6-v2 embeds email content into 384-dimensional vectors stored in ChromaDB collections. At query time, cosine similarity retrieves the top-k most relevant chunks as context for Gemini quiz generation. The RAG approach grounds quiz questions in real data — preventing hallucinated or irrelevant questions.

API Documentation Report The FastAPI Swagger UI at localhost:8000/docs serves as a live, auto-generated API reference report. It documents all 13 endpoints with full request body schemas, response models, authentication requirements, query parameter descriptions, and interactive test execution — making it the single source of truth for all API contracts in the system.

3. Key Outcomes

- Successfully deployed a complete production-grade microservices platform on AWS EC2, accessible worldwide at http://13.206.119.123:8080 — demonstrating end-to-end DevOps from source code to live cloud application.
- Fully automated CI/CD pipeline running on GitHub Actions — 3 jobs, 7 minutes, zero manual steps — building and pushing 5 Docker images on every code commit. Run #7 confirmed: all jobs GREEN.
- Terraform IaC provisions a complete AWS environment (9 resources including VPC, EC2, Elastic IP) from scratch in under 5 minutes — infrastructure is version-controlled, repeatable, and auditable.
- K3s Kubernetes running on EC2 with 10 manifests deployed. Live scaling demonstrated: triage-backend scaled from 1 to 3 replicas in under 15 seconds with a single kubectl command.
- 5 Docker images published to Docker Hub (anushkakashyap31/supporthub-*) — publicly accessible and pullable by any environment. Multi-stage builds reduce frontend image size by ~94%.
- TriageAgent 4-agent AI pipeline processing messages in 5–10 seconds — a 10–20× improvement over 15–20 minute manual triage. Deployed live on AWS with Firebase Firestore persistence.
- QuizBot RAG-based quiz generation running live on AWS — ChromaDB vector store, Sentence-Transformers embeddings, and Gemini-powered MCQ generation all operational in the same Docker container.
- Proper secret management implemented throughout — GitHub Secrets for CI/CD, .env files for runtime config, gitignored Firebase credentials — no sensitive data ever committed to version control.
- Complete DevOps toolchain demonstrated: Git + GitHub + Docker + Docker Compose + GitHub Actions + Docker Hub + Terraform + AWS EC2 + K3s Kubernetes + Nginx — all integrated in a single working system.

Conclusion

SupportHub successfully demonstrates how industry-standard DevOps practices can be applied to real, production-grade AI applications — transforming two independently developed microservices into a unified, cloud-deployed, fully automated platform. The project fulfills every requirement of the DO-15 problem statement — Infrastructure as Code with Terraform, containerized deployment with Docker, automated CI/CD with GitHub Actions, cloud deployment on AWS EC2, and container orchestration with Kubernetes — while simultaneously delivering two functional AI applications that solve a genuine problem for non-profit organizations.

The architectural decisions made throughout the project reflect industry best practices. The microservices separation between TriageAgent and QuizBot ensures that they can scale, fail, and be updated independently. The multi-stage Docker builds minimize image sizes and eliminate unnecessary runtime dependencies. The GitHub Actions pipeline eliminates manual deployment errors and creates a complete audit trail of every build. Terraform IaC ensures that the entire AWS infrastructure can be recreated from scratch.

One of the most significant technical challenges overcome during this project was the dependency resolution for QuizBot's quiz-backend. The sentence-transformers library requires a specific version of huggingface-hub that conflicts with transformers' requirements. After systematic debugging of pip dependency trees, the working combination of sentence-transformers==2.2.2, huggingface-hub==0.21.4, and transformers==4.30.2 was identified and pinned — alongside the deliberate exclusion of unnecessary heavy packages (pymongo, faiss-cpu, torchvision) that were causing disk space exhaustion on the EC2 server. This real-world debugging experience demonstrates the complexity of production Python dependency management.

The port conflict between K3s Kubernetes and Docker Compose (both attempting to use port 80) led to the discovery that K3s's built-in ingress controller intercepts port 80 traffic before Docker's Nginx container can receive it. The solution — changing Docker Compose nginx to port 8080 — demonstrates practical knowledge of how containerization and Kubernetes interact with the host network stack. Similarly, the quizbot.db SQLite file being created as a directory by Docker (when the bind mount target doesn't exist) required understanding of how Docker handles volume mount creation, fixed by pre-creating the file with touch before starting containers.

The WSL2 memory configuration (.wslconfig with 6GB memory, 4 processors, 4GB swap) for Docker Desktop on Windows, and the RAM constraint management on the t3.small EC2 instance (never running Docker Compose and K3s simultaneously), demonstrate the practical infrastructure awareness required for real-world DevOps work — understanding not just the tools but the resource constraints they operate within.

From a learning perspective, this project provided comprehensive hands-on experience across the complete DevOps toolchain. Writing Terraform HCL to provision AWS resources, debugging GitHub Actions YAML workflow syntax, understanding Docker multi-stage build optimizations, configuring Kubernetes liveness probes and resource limits, managing K3s on a RAM-constrained server, and troubleshooting Python dependency conflicts — each of these experiences represents a skill directly applicable to professional DevOps engineering roles.

The two AI applications themselves — TriageAgent's 4-agent LangChain pipeline and QuizBot's ChromaDB RAG system — demonstrate that modern AI application development requires not just algorithmic thinking but robust engineering practices. Running these applications in production requires proper containerization, secret management, health checks, and graceful error handling — all of which were implemented and validated in this project.

In conclusion, SupportHub is not merely a demonstration project — it is a deployable, maintainable, and extensible platform that combines the full spectrum of modern DevOps practices with real AI-powered functionality. The platform is live, accessible, and ready to serve as the foundation for further development, making it a complete and successful outcome for the DO-15 course requirement.

Key Learnings:

- Hands-on experience with the complete DevOps toolchain: Git, Docker, GitHub Actions, Terraform, AWS EC2, K3s Kubernetes, Nginx — all integrated in a single working system.
- Practical understanding of Docker multi-stage builds, container networking, volume mounts, and docker-compose orchestration for multi-service applications.
- Real-world CI/CD pipeline implementation using GitHub Actions — 3-job workflow with secret management, Docker Buildx, and automated registry publishing.
- Terraform IaC for AWS — provisioning VPCs, subnets, security groups, EC2 instances, and Elastic IPs as version-controlled, repeatable infrastructure code.
- Kubernetes deployment and operations: manifest writing, pod management, service types (ClusterIP vs NodePort), liveness probes, resource limits, and live scaling demonstrations.
- Python dependency management in Docker — resolving version conflicts between sentence-transformers, huggingface-hub, and transformers through systematic debugging of pip dependency trees.

Future Scope & Enhancements

As a v1.0 platform running on a single EC2 instance, SupportHub has a strong DevOps and AI foundation ready for professional-grade enhancements. The following improvements are planned for future versions:

- HTTPS/SSL with Custom Domain: Configure an Application Load Balancer (ALB) with an ACM SSL certificate and a custom domain (e.g., supporthub.example.org) — replacing the current HTTP-only EC2 IP access with secure HTTPS access. This is the most critical production readiness improvement.
- Terraform Remote State: Move Terraform state from a local .tfstate file to an S3 backend with DynamoDB state locking — enabling collaborative infrastructure management and preventing state corruption from concurrent applies.
- Auto-Scaling Group: Replace the single EC2 instance with an Auto Scaling Group (ASG) behind a Load Balancer — automatically adding EC2 instances during peak load and removing them during quiet periods, reducing cost while improving availability.
- Kubernetes Horizontal Pod Autoscaler (HPA): Add HPA manifests that automatically scale pods based on CPU and memory metrics — removing the need for manual kubectl scale commands and enabling truly automated capacity management.
- Separate Namespaces: Create separate Kubernetes namespaces for triage, quizbot, and infrastructure services — providing better resource isolation, access control, and easier monitoring per application.
- Centralized Logging with ELK Stack: Deploy Elasticsearch, Logstash, and Kibana (ELK) on a separate instance to aggregate logs from all 5 containers — replacing per-container docker logs with searchable, filterable, centralized log management.